ETSIIT
Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

Ingeniería Informática + ADE

Universidad de Granada (UGR)

Autor: Ismael Sallami Moreno

Asignatura: Apuntes Sistemas Concurrentes y Distribuidos (SCD)

Índice

1. Tema 3: Sistemas basados en el paso de mensajes **3**
1.1. El cuello de botella de Von Neumann . . . 3
1.2. Clasificación de Flynn . . . 3
1.3. Diferencias entre Multiprocesador SMP y AMP . . . 5
1.4. Mecanismo de citas . . . 5
1.5. Paso de mensajes bloqueante con búfer . . . 5
1.6. Paso de mensajes no bloqueante con búfer . . . 6
1.7. MPI . . . 6
1.7.1. Comunicación bloqueante . . . 6
1.7.2. Comunicación No bloqueante . . . 6
1.7.3. Sondeo de Mensajes . . . 6
1.8. Explicación del código de la criba de Eratóstenes . . . 7
1.8.1. Explicación del problema del museo . . . 7
1.9. Orden Select . . . 8
1.9.1. Semántica de la orden select . . . 8
1.9.2. Formas de poner la guarda de una orden select . . . 9
1.9.3. Tipos de ejecución de las guardas . . . 9
1.9.4. Determinación de la guarda a ejecutar . . . 9
1.9.5. Ejemplo: Productor-Consumidor con Búfer FIFO . . . 9
1.9.6. Select con Guardas Indexadas . . . 11
1.9.7. Select con sentencia else . . . 12

2. Tema 4: Introducción a los Sistemas de Tiempo Real **13**
2.1. Confusión con otros sistemas . . . 13
2.2. Propiedades de los STR . . . 13
2.3. Definición en el ámbito de los sistemas operativos . . . 14
2.4. Clasificación de los STR . . . 14
2.4.1. Atendiendo a la criticidad de los STR . . . 14
2.5. Tipos de medidas del tiempo de interés en STR . . . 15
2.5.1. 1. Tiempo Absoluto . . . 15
2.5.2. 2. Intervalos o Tiempo Relativo . . . 15
2.5.3. Concepto de reloj de tiempo real . . . 16
2.5.4. Características más importantes de los relojes de tiempo real . . . 16
2.5.5. Escalas temporales . . . 16
2.5.6. Precisión y Intervalo para un contador de 32 bits . . . 17

1 Tema 3: Sistemas basados en el paso de mensajes

1.1. El cuello de botella de Von Neumann

El **cuello de botella de Von Neumann** se refiere a una limitación inherente en las arquitecturas de computación clásicas basadas en el modelo de programa único almacenado. Este problema surge debido a la dependencia de un único bus para transferir datos e instrucciones entre la memoria y la unidad de procesamiento central (CPU). A continuación, se presentan sus principales características y consecuencias:

- **Limitación de velocidad del bus:** La transferencia de datos entre la memoria y la CPU está restringida por la velocidad máxima del bus, lo que afecta el rendimiento general del sistema.
- **Cuello de botella:** Dado que no es posible realizar simultáneamente una operación de búsqueda de instrucciones y una operación de datos, se genera un retraso en el flujo de ejecución de las instrucciones.
- **Impacto en el rendimiento:** Esta arquitectura requiere más tiempo de ejecución para los programas, lo que reduce la eficiencia del sistema computacional.

El cuello de botella de Von Neumann es una de las razones principales por las cuales las arquitecturas modernas buscan soluciones más eficientes, como los sistemas multiprocesadores o arquitecturas paralelas.

1.2. Clasificación de Flynn

La **clasificación de Flynn** es un esquema que categoriza las arquitecturas de sistemas computacionales en función de cómo manejan las instrucciones y los datos durante la ejecución. Esta clasificación es fundamental para entender los diferentes modelos de procesamiento y se divide en cuatro categorías principales:

- **SISD (Single Instruction, Single Data):**
  - Una sola unidad de control ejecuta una única secuencia de instrucciones.
  - Procesa un único flujo de datos.
  - Ejemplo: computadores monoprocesador tradicionales.
- **SIMD (Single Instruction, Multiple Data):**
  - Una única instrucción se aplica simultáneamente a múltiples flujos de datos.
  - Común en aplicaciones con operaciones repetitivas sobre grandes conjuntos de datos, como gráficos y procesamiento de señales.
  - Ejemplo: procesadores vectoriales y GPU.
- **MISD (Multiple Instruction, Single Data):**

- Múltiples unidades de control ejecutan distintas instrucciones sobre un único flujo de datos.
- Es menos común en sistemas reales debido a limitaciones prácticas.

- **MIMD (Multiple Instruction, Multiple Data):**
  - Múltiples unidades de control ejecutan distintas secuencias de instrucciones sobre múltiples flujos de datos.
  - Flexible y utilizado en sistemas paralelos modernos.
  - Ejemplo: arquitecturas multicore y clusters de computadoras.

Esta clasificación permite identificar las capacidades y limitaciones de las diferentes arquitecturas, sirviendo como guía para diseñar y optimizar sistemas computacionales.

1.3. Diferencias entre Multiprocesador SMP y AMP

Características	**SMP (Multiprocesamiento Simétrico)**	**AMP (Multiprocesamiento Asimétrico)**
Arquitectura	Todos los procesadores son iguales y comparten el acceso a la memoria	Un procesador principal controla el sistema y los otros son secundarios
Acceso a memoria	Todos los procesadores tienen acceso compartido a la memoria central	Solo el procesador principal tiene acceso a la memoria central
Control de procesos	Los procesadores pueden trabajar independientemente	El procesador principal gestiona los procesos y asigna tareas a los secundarios
Escalabilidad	Alta escalabilidad; se pueden agregar más procesadores sin mucha complicación	Baja escalabilidad; añadir más procesadores no mejora el rendimiento de forma eficiente
Costo	Más costoso debido a la necesidad de múltiples procesadores iguales y una memoria compartida	Menos costoso, ya que solo se necesita un procesador principal y algunos secundarios
Rendimiento	Mejor rendimiento en tareas paralelas y de procesamiento intensivo	Menor rendimiento en tareas paralelas, debido a la dependencia del procesador principal
Sistemas típicos	Servidores, estaciones de trabajo y sistemas de alta gama	Sistemas embebidos, estaciones de trabajo de menor escala y dispositivos con recursos limitados

Cuadro 1: Diferencias entre Multiprocesador SMP y AMP

1.4. Mecanismo de citas

Se trata de una operación de comunicación NO bloqueante y sin búfer. La cita tiene lugar antes de que comience la transmisión de datos.

1.5. Paso de mensajes bloqueante con búfer

El proceso emisor vuelve inmediatamente al ejecutar la operación de envío, *salvo que el proceso buffer esté lleno*. En este caso, el proceso emisor se bloquea hasta que haya espacio en el buffer. La operación *receive* no vuelve hasta que se han recibido los datos.

1.6. Paso de mensajes no bloqueante con búfer

En este caso se reduce el tiempo de espera debido a que la operación *receive* provoca la transferencia inmediata de datos del búfer a la memoria del proceso receptor.

1.7. MPI

1.7.1. Comunicación bloqueante

Campos de operación MPI de envío con buffer

```
int MPI_Send(void *buf, int count, MPI_Datatype datatype, int
   dest, int tag, MPI_Comm comm);
```

Campos de operación MPI de recepción con buffer

```
int MPI_Recv(void *buf, int count, MPI_Datatype datatype, int
   source, int tag, MPI_Comm comm, MPI_Status *status);
```

- El *origen, tag, comunicador* deben de coincidir en el mensaje enviado y recibido.
- *MPI_Send, MPI_Recv, MPI_Ssend* no vuelven hasta que se completan.

1.7.2. Comunicación No bloqueante

- *MPI_Isend, MPI_Irecv, MPI_Iprobe, MPI_Test* son operaciones no bloqueantes.
  - *MPI_Isend:* Inicia una operación de envío no bloqueante. Permite que el programa continúe ejecutándose mientras se realiza la operación de envío en segundo plano.
  - *MPI_Irecv:* Inicia una operación de recepción no bloqueante. Permite que el programa continúe ejecutándose mientras se realiza la operación de recepción en segundo plano.
  - *MPI_Iprobe:* Permite comprobar de manera no bloqueante si hay un mensaje disponible para ser recibido. Esto es útil para evitar que el programa se bloquee esperando un mensaje.
  - *MPI_Test:* Verifica si una operación no bloqueante ha completado. Esto permite al programa comprobar el estado de las operaciones no bloqueantes y actuar en consecuencia.

1.7.3. Sondeo de Mensajes

- *MPI_Iprobe* y *MPI_Probe* son operaciones de sondeo de mensajes que permiten a un proceso verificar si hay mensajes disponibles para ser recibidos.

- *MPI_Iprobe:* Permite sondear de manera no bloqueante si hay mensajes disponibles para ser recibidos. Devuelve un valor verdadero si hay mensajes pendientes y falso en caso contrario. El valor que modifica es el flag, es decir, si flag>0, hay que recibirlo con MPI_Recv, debido a que se da la existencia de un mensaje no bloqueante.
- *MPI_Probe:* Permite sondear de manera bloqueante si hay mensajes disponibles para ser recibidos. Se bloquea hasta que llega un mensaje.

Diferencias entre MPI_Probe y MPI_IProbe

Característica	**MPI_Probe**	**MPI_IProbe**
Bloqueante	Sí	No
Comportamiento	Espera hasta que haya un mensaje disponible	Vuelve inmediatamente con el estado del mensaje
Eficiencia	Puede causar ineficiencia en ausencia de mensajes mientras se espera	Permite trabajar en paralelo mientras se espera

Cuadro 2: Diferencias entre MPI_Probe y MPI_IProbe

1.8. Explicación del código de la criba de Eratóstenes

Para ello pincha aquí. Además la actividad extra se encuentra aquí.

1.8.1. Explicación del problema del museo

```
    PUERTA(i:1..2)::
{ int s=0;
do
if (s<HORA.CIERRE && PERSONA())->
{send(CONTROL, S()); //envia una se\~nal de entrada
de persona
DELAY.UNTIL(s+1);//espera hasta el siguiente
instante
s:=s+1;// cuenta un nuevo tick de reloj
}
[]
(s<HORA.CIERRE && NOT PERSONA())->
DELAY.UNTIL(s+1);//espera hasta el siguiente
instante
s:=s+1;]//cuenta otro tick
[]
TRUE->DELAY.UNTIL (TIME() + 16*3600);
//es la hora de cierre del museo; hay
//que esperar 16 horas para activar el controlador
.
// TIME() devuelve una cuenta en segundos.
fi
```

```
do;
send(CONTROL, Start();}
CONTROL::
{ int cont= 0;
if receive(PUERTA(1), Start());
//desde cualquiera de los sensores
[] //de las puertas se arranca
receive(PUERTA(2), Start());//el controlador
fi
do
[]*[(j:1..2) receive(PUERTA(j), S())-> cont:= cont
+1];
//cuenta una persona mas, porque ha recibido la se\~nal
//de cualquiera de las 2 puertas (no se puede saber cual)
od;
printf("numero de personas", %d, cont));
}
main(){
cobegin PUERTA;CONTROL coend;}
```

1.9. Orden Select

- versión de orden alternativa no determinista
- modo síncrono de comunicación, paso de mensajes bloqueantes
- resuelve el problema de la recepción de tener varias variables pendientes sin dependencias del orden temporal

1.9.1. Semántica de la orden select

- cada bloque que comienza con *when* se denomina alternativa (orden guarda)
- desde when hasta do se denomina guarda de dicha alternativa
- instrucciones receive nombran a otros procesos del programa concurrente y cada uno referencia a una variable local

```
    select
when condicion1 receive( variable1, proceso1 ) do
sentencias1
when condicion2 receive( variable2, proceso2 ) do
sentencias2
...
when condicionn receive( variablen, proceson ) do
sentenciasn
end
```

1.9.2. Formas de poner la guarda de una orden select

- when receive(mensaje, proceso) do sentencias
- when true receive(mensaje, proceso) do sentencias
- when condicion do sentencias (puede omitirse el receive, decimos que es una guarda sin sentencia de entrada)

1.9.3. Tipos de ejecución de las guardas

- guarda ejecutable: si la condición es verdadera y ya ha iniciado la sentencia send
- guarda potencialmente ejecutable: si la condición es verdadera pero no ha iniciado la sentencia send
- guarda no ejecutable

1.9.4. Determinación de la guarda a ejecutar

- aquella que inicio el send antes
- se selecciona no determinísticamente una cualquiera si no hay guardas con sentencias de entrada
- si hay solo guardas potenciablemente ejecutables, se inicia la guarda cuando esta inicia la sentencia send
- si no hay ningún tipo de guarda se genera una excepción/error.

Hay que tener en cuenta que la ejecución de un instrucción select conlleva esperas, por lo que pueden producirse situaciones de interbloqueo.

Existe una orden select con prioridad, por lo que dejaría de ser no determinística.

Para programar a un servicio servidor se debe de ejecutar el select dentro de un bucle para que se evalúe de nuevo la guarda y se seleccione no determinísticamente a una de ellas para poder ejecutarlas.

1.9.5. Ejemplo: Productor-Consumidor con Búfer FIFO

Este ejemplo ilustra el patrón clásico de **productor-consumidor** utilizando un búfer FIFO intermedio. En este esquema:

- El **productor** genera elementos y los envía al búfer.
- El **intermedio** actúa como un búfer que controla la inserción y extracción de datos, garantizando condiciones de sincronización mediante guardas.
- El **consumidor** extrae elementos del búfer y los procesa.

La clave de este diseño radica en que el intermedio no conoce de antemano el orden de las peticiones de inserción y extracción, pero asegura la propiedad de **seguridad** en el acceso concurrente al búfer.

```
{ Productor (P) }
while true do                          // Inicia un bucle infinito en el productor.
begin
    v := Produce();                    // El productor genera un valor 'v'.
    s_send(v, B);                      // El productor envía el valor 'v' al búfer 'B'.
end

{ Intermedio (B) }
var esc, lec, cont: integer := 0;      // Inicializa las variables: 'esc' (índice de escritura), 'lec' (índice de lectura) y 'cont' (contador de elementos en el búfer).
buf: array[0..tam-1] of integer;       // Declara el búfer 'buf' como un arreglo de tamaño 'tam', donde se almacenarán los valores producidos.
begin
    while true do                      // Inicia un bucle infinito en el intermedio (el búfer).
        select                         // Comienza una selección de condiciones para ser ejecutadas.
            when cont < tam receive(v, P) do       // Si el número de elementos en el búfer es menor que 'tam', hay espacio para almacenar más elementos, El intermedio recibe un valor 'v' del productor (P).

                buf[esc] := v;         // El valor recibido se almacena en la posición 'esc' del búfer.
                esc := (esc + 1) mod tam;  // Se actualiza el índice de escritura 'esc' de manera circular (se vuelve a 0 cuando alcanza 'tam').
                cont := cont + 1;      // Se incrementa el contador de elementos en el búfer.
            when 0 < cont receive(s, C) do             // Si el número de elementos en el búfer es mayor que 0, hay datos para consumir, El intermedio recibe una solicitud del consumidor (C).

                s_send(buf[lec], C);   // El intermedio envía el valor almacenado en 'lec' al consumidor.
                lec := (lec + 1) mod tam;  // Se actualiza el índice de lectura 'lec' de manera circular.
                cont := cont - 1;      // Se decrementa el contador de elementos en el búfer.
        end
end

{ Consumidor (C) }
while true do                          // Inicia un bucle infinito en el consumidor.
begin
```

```
        s_send(s, B);                    // El consumidor solicita un valor
            del búfer (envía una solicitud 's' al búfer).
        receive(v, B);                   // El consumidor recibe el valor '
           v' del búfer.
        Consume(v);                      // El consumidor procesa el valor
           recibido.
    end
```

Listing 1: Productor-Consumidor con Búfer FIFO

Breve Explicación

- **Productor (P):** Un bucle infinito genera valores (*Produce*) y los envía al búfer (*s_send(v, B)*).
- **Intermedio (B):** Administra las operaciones de inserción y extracción usando un arreglo cíclico:
  - Inserta valores en el búfer si no está lleno (*cont <tam*).
  - Extrae valores del búfer si no está vacío (*cont >0*).
  - Las guardas en las sentencias *select* controlan estas condiciones.
- **Consumidor (C):** Recibe valores del búfer y los consume (*Consume(v)*).

Este modelo asegura la sincronización entre los procesos mediante las condiciones de las guardas, evitando inconsistencias en el acceso concurrente al búfer.

1.9.6. Select con Guardas Indexadas

Instrucción `Select` con guardas indexadas Cuando es necesario replicar alternativas en una estructura de espera selectiva, se puede utilizar una construcción que evita la redundancia de código. La sintaxis para lograrlo es:

```
for indice := inicial to final
when condicion receive(mensaje, proceso) do
    sentencias(indice);
```

En esta construcción, tanto la condición, el mensaje, el proceso y las sentencias pueden referirse al índice de la iteración. Esta forma compacta es equivalente a expandir explícitamente las alternativas, como se muestra a continuación:

```
when condicion receive(mensaje, proceso) do
    sentencias { se sustituye indice por inicial }
when condicion receive(mensaje, proceso) do
    sentencias { se sustituye indice por inicial + 1 }
...
```

Este mecanismo facilita la creación de programas distribuidos más legibles y eficientes, reduciendo la redundancia en el código fuente.

1.9.7. Select con sentencia else

En este caso es similar al uso del *when*, pero en este caso debemos de hacer uso del *else*, podemos concebir que el uso y desempeño es similar al de in ifelse, pero en este caso es un whenelse. Ejemplo:

```
    // Declaración de variables globales
    var suma : array[0..n-1] of integer := (0,0,...,0) ; // Arreglo para almacenar las sumas parciales de cada proceso.
    continuar : boolean := true ;                        // Variable de control para el bucle principal.
    numero : integer ;                                   // Variable para recibir el número enviado por un proceso.

    begin
    while continuar do begin                             // Bucle principal que se ejecuta mientras 'continuar' sea verdadero.
        select                                            // Inicia la estructura de selección para manejar múltiples condiciones.
            for i := 0 to n - 1                          // Itera sobre todos los procesos del 0 al n-1.
            when suma[i] < 100 receive( numero, emisor[i] ) // Si la suma parcial del proceso i es menor que 100,
                                                                // recibe un número de dicho proceso (emisor[i]).
            do
                suma[i] := suma[i] + numero ;            // Actualiza la suma parcial del proceso i con el número recibido.
                continuar := true ;                      // Establece 'continuar' como verdadero para seguir ejecutando el bucle.
                                                          // Nota: esta instrucción no es estrictamente necesaria, pero aclara
                                                          // que el bucle continuará.
        end
        else continuar := false;                         // Si no se cumple ninguna condición, establece 'continuar' como falso
                                                         // para salir del bucle principal.
    end
    end
```

Listing 2: Select con sentencia else

2 Tema 4: Introducción a los Sistemas de Tiempo Real

2.1. Confusión con otros sistemas

Los sistemas de tiempo real (STR) a menudo se confunden con otros tipos de sistemas debido a ciertas similitudes en sus características y comportamientos. A continuación, se explican las razones de estas confusiones:

- **En línea:** Los sistemas en línea están diseñados para estar disponibles y operativos en todo momento, similar a los STR que deben responder a eventos en tiempo real. Sin embargo, la diferencia clave es que los STR tienen restricciones temporales estrictas que deben cumplirse para garantizar el correcto funcionamiento del sistema.

- **Interactivos:** Los sistemas interactivos permiten la interacción directa con los usuarios, lo que puede dar la impresión de que son sistemas de tiempo real. No obstante, los STR no solo requieren interacción, sino que también deben cumplir con plazos específicos para procesar y responder a eventos.

- **Rápidos:** La velocidad de respuesta es una característica común tanto en sistemas rápidos como en STR. Sin embargo, la principal diferencia radica en que los STR no solo deben ser rápidos, sino que deben garantizar que las respuestas ocurran dentro de un tiempo predefinido y crítico para el correcto funcionamiento del sistema.

2.2. Propiedades de los STR

Los Sistemas de Tiempo Real (STR) tienen ciertas propiedades fundamentales que los distinguen de otros sistemas informáticos. Estas propiedades son esenciales para asegurar que los STR cumplan con los requisitos temporales y de fiabilidad en su funcionamiento. Las principales propiedades de los STR son las siguientes:

- **Reactividad:** Los sistemas de tiempo real deben reaccionar de manera adecuada y rápida a los eventos que ocurren en el entorno. La reactividad implica que el sistema debe responder a los eventos en un tiempo predecible, es decir, la respuesta no solo depende de lo que el sistema haga, sino también de cuándo lo haga. Esta propiedad es crucial en aplicaciones como el control de sistemas de vuelo o los sistemas de frenado en vehículos.

- **Determinismo:** Un STR debe ser determinista, lo que significa que dada una entrada específica, el sistema debe producir siempre la misma salida dentro de un tiempo determinado. El comportamiento de un STR es predecible y se puede calcular con antelación. Esto es fundamental en sistemas como los de control de maquinaria, donde la predictibilidad es crucial para garantizar el buen funcionamiento del sistema sin errores inesperados.

- **Responsabilidad:** Esta propiedad hace referencia a la capacidad de un STR para garantizar que las tareas o procesos se completan dentro de sus plazos establecidos. En otras palabras, un STR debe ser capaz de asegurar que las tareas se

ejecuten correctamente y a tiempo, incluso bajo condiciones de carga alta o en situaciones imprevistas. La responsabilidad es vital para garantizar la estabilidad y efectividad del sistema.

- **Confiabilidad:** Los STR deben ser confiables, lo que significa que deben funcionar correctamente y de forma estable durante su operación, incluso ante fallos o errores. La confiabilidad es una propiedad clave para sistemas críticos como los utilizados en aplicaciones médicas, aeronáuticas o automotrices, donde un fallo podría tener consecuencias graves. La fiabilidad implica que los sistemas estén diseñados para manejar fallos de manera segura o minimizar su impacto.

2.3. Definición en el ámbito de los sistemas operativos

Sistema Operativo de Tiempo Real es aquel que tiene la capacidad para suministrar un nivel de servicio requerido en un tiempo limitado y especificado a priori.

2.4. Clasificación de los STR

Los Sistemas de Tiempo Real (STR) se pueden clasificar según el nivel de criticidad temporal que tienen sus tareas, lo cual influye en los requisitos y características de cada sistema. A continuación, se presentan tres categorías basadas en la criticidad de los STR:

2.4.1. Atendiendo a la criticidad de los STR

- **Misión Crítica:** Estos sistemas son los más críticos, ya que cualquier fallo en la ejecución dentro del tiempo establecido puede tener consecuencias graves. Un ejemplo de este tipo de sistema es el *control de aterrizaje* de aeronaves, donde la puntualidad y exactitud son cruciales para la seguridad de la operación. Los complementos importantes en estos sistemas incluyen *tolerancia a fallos*, lo que significa que el sistema debe ser capaz de manejar y recuperarse de errores sin comprometer la seguridad ni la efectividad.

- **Estrictos:** Los sistemas estrictos también requieren una alta precisión temporal, aunque las consecuencias de un fallo no son tan críticas como en los sistemas de misión crítica. Un ejemplo sería el sistema de *reservas de vuelos*, donde las respuestas deben ser rápidas y dentro de los tiempos previstos, pero un pequeño retraso no implica necesariamente un fallo catastrófico. Los complementos asociados a estos sistemas incluyen *calidad de la respuesta*, ya que es importante que la respuesta no solo sea puntual, sino que también sea precisa y eficiente.

- **Permisivos:** Estos sistemas tienen requisitos menos estrictos en cuanto a tiempo, ya que las tareas pueden tolerar ciertos retrasos sin consecuencias graves. Un ejemplo típico sería la *adquisición de datos meteorológicos*, donde los datos pueden ser procesados con algo de retraso sin que afecte gravemente al resultado final. Los complementos de estos sistemas incluyen *medidas de fiabilidad*, lo que significa que, aunque el sistema no necesite ser extremadamente puntual, debe ser confiable y eficiente en su funcionamiento.

2.5. Tipos de medidas del tiempo de interés en STR

Existen dos tipos principales de medidas del tiempo de interés en los Sistemas de Tiempo Real (STR): el **tiempo absoluto** y los **intervalos o tiempo relativo**. A continuación se detallan ambos tipos:

2.5.1. 1. Tiempo Absoluto

El tiempo absoluto se refiere a la medición del tiempo con relación a un sistema de referencia global. Algunos de los sistemas de referencia más comunes incluyen:

- **Sistemas de referencia locales:** Este tipo de medida usa un sistema de tiempo basado en la localización geográfica de un lugar específico. Puede estar basado en relojes locales o sistemas de referencia regionales.
- **Astronómicos (UT0):** Se refiere al tiempo universal basado en la rotación de la Tierra en relación con las estrellas. El *UT0* es un tiempo que considera los movimientos irregulares de la Tierra y es usado como una medida precisa en astronomía.
- **Atómicos (IAT):** El Tiempo Atómico Internacional (IAT) es una medida basada en la oscilación de los átomos de cesio y es utilizado para obtener un estándar muy preciso de medición del tiempo. Este tiempo es utilizado como base para el UTC.
- **Tiempo Coordinado Universal (UTC):** Es el estándar internacional de tiempo utilizado para coordinar el tiempo globalmente. Desde 1972, el UTC ha sido utilizado ampliamente y es una medida combinada basada en el IAT, pero con ajustes ocasionales para sincronizarlo con la rotación de la Tierra.
- **Satelital (GPS):** El tiempo de GPS (GPST) es una escala de tiempo continua que no tiene segundos intercalares. Se define a partir del segmento de control del sistema GPS, el cual se basa en relojes atómicos ubicados en estaciones de monitoreo y en los satélites. Este sistema comenzó a las 00:00 UTC del 5 al 6 de enero de 1980.

2.5.2. 2. Intervalos o Tiempo Relativo

El tiempo relativo se refiere a la medición de intervalos de tiempo entre eventos, y no a un instante específico en un sistema de referencia global. Los intervalos de tiempo pueden no ser monótonos, lo que significa que pueden verse afectados por factores como la deriva o los ajustes de reloj. En STR, esto es importante porque el tiempo computacional puede no ser uniforme o estable debido a la interferencia de diferentes tareas en ejecución.

Por ejemplo, el *Tiempo Computador* puede variar dependiendo de la carga del sistema, lo que hace que el tiempo sea no monótono, y este comportamiento debe ser gestionado adecuadamente para garantizar el cumplimiento de los plazos de las tareas en los STR.

La medida del tiempo en los Sistemas de Tiempo Real (STR) se realiza a través de diferentes tipos de relojes y características que permiten un seguimiento preciso de los eventos y tareas dentro de los sistemas. A continuación se presentan los conceptos clave y características de los relojes de tiempo real:

2.5.3. Concepto de reloj de tiempo real

El reloj de tiempo real puede estar basado en diferentes componentes que permiten su funcionamiento:

- **Oscilador:** Es un dispositivo que genera una señal de frecuencia constante, utilizada para medir el tiempo. Los osciladores suelen ser muy precisos, pero pueden tener limitaciones en cuanto a estabilidad a largo plazo.
- **Contador:** Un contador de tiempo real lleva el registro del tiempo mediante una secuencia de pulsos. Cada pulso representa una unidad de tiempo y el contador se incrementa con cada uno. Este tipo de reloj puede ser más eficiente en cuanto a precisión temporal.
- **Software:** Los relojes de tiempo real basados en software utilizan el sistema operativo y el hardware subyacente para realizar mediciones temporales. Estos pueden estar sujetos a variaciones o retardos dependiendo de las cargas de trabajo del sistema.

2.5.4. Características más importantes de los relojes de tiempo real

Los relojes de tiempo real tienen características fundamentales que permiten su funcionamiento adecuado en los STR:

- **Precisión (granularidad):** La precisión de un reloj de tiempo real se refiere a la mínima unidad de tiempo que puede medir. Esta se mide generalmente en nanosegundos (ns), microsegundos (μs), milisegundos (ms), o segundos (s). Cuanto mayor sea la precisión, mayor será la capacidad del sistema para gestionar tareas con plazos muy estrictos.
- **Tiempo de desbordamiento:** El tiempo de desbordamiento se refiere al límite de tiempo después del cual un contador o reloj vuelve a cero. Este comportamiento es común en relojes de 32 bits, ya que al llegar al límite de valor, el contador se reinicia.
- **Intervalo:** El intervalo se refiere al tiempo transcurrido entre dos eventos consecutivos o dos mediciones del reloj. Este intervalo puede ser afectado por la precisión del reloj y la estabilidad del sistema.

2.5.5. Escalas temporales

Existen varias escalas temporales utilizadas en los STR, dependiendo del tipo de medición y de las necesidades de la aplicación. Algunas de las más comunes son:

- **Tiempo monótono:** Es un tipo de tiempo en el que el reloj avanza de manera continua, sin retroceder o saltar en el tiempo. Es muy útil para aplicaciones que requieren un seguimiento constante del tiempo.
- **Tiempo no monótono:** En este caso, el reloj puede avanzar o retroceder en función de varios factores, como ajustes de los relojes o cambios de sistema. Es común en sistemas que necesitan sincronización con otros sistemas o eventos externos, como los relojes GPS.
- **Tiempo absoluto:** Es un tiempo basado en un sistema de referencia global (por ejemplo, UTC), que no depende de los eventos o el sistema local.
- **Tiempo no absoluto:** El tiempo no absoluto depende del sistema local y puede estar sujeto a variaciones o desincronización respecto a un sistema de referencia global.

2.5.6. Precisión y Intervalo para un contador de 32 bits

A continuación se muestra la relación entre la precisión y el intervalo para un contador de 32 bits. Este tipo de contador tiene una capacidad limitada debido al tamaño del número que puede almacenar (32 bits), lo que implica que el contador se desbordará después de un determinado intervalo.

- **Precisión:** La precisión de un contador de 32 bits puede variar dependiendo de la unidad de medida. A continuación se presentan algunos ejemplos de intervalos de tiempo que un contador de 32 bits puede manejar:
  - 100 ns (nanosegundos) hasta 429,5 segundos.
  - 1 μs (microsegundos) hasta 71,58 minutos.
  - 100 μs (microsegundos) hasta 119,3 horas.
  - 1 ms (milisegundos) hasta 49,71 días.
  - 1 s (segundo) hasta 136,18 años.

  Estos valores indican el intervalo máximo que un contador de 32 bits puede medir antes de que se produzca un desbordamiento. Cuanto mayor sea la precisión, menor será el intervalo que el contador podrá gestionar antes de desbordarse.